

Dekoratory i Metaprogramowanie

Autorzy:
Michał Sałatpat
Krzysztof Słowikowski (FEST)



Dekoratory

(prezentacja może zawierać dekoracje)

Czym są dekoratory?

- Dekorator to funkcja, która dostaje funkcję jako argument i zwraca nową funkcję (opakowaną w kod)
- Dekoratory są bezpośrednim zastosowaniem koncepcji [programowania funkcyjnego](#)
- Wprowadzone w pythonie 2.4 (2004 rok)
- Główne zadanie – redukcja powtarzalnego kodu
- Główne zadanie 2 – separacja odpowiedzialności



```
def dekorator(funkcja):  
    def opakowanie():  
        print("Najpierw to")  
        funkcja()  
        print("To na końcu")  
    return opakowanie
```

```
@dekorator  
def jakas_funkcja():  
    print("jakis tekst")  
  
jakas_funkcja()
```



Struktura Dekoratora (najprostszego)



```
def dekorator(funkcja):  
    def opakowanie():  
        print("Najpierw to")  
        funkcja()  
        print("To na końcu")  
    return opakowanie
```

Fukcja
wewnętrzna

Dekorator
zwraca
funkcję, jako
obiekt

Znak '@' mówi
pythonowi by
zastosował
dany dekorator
do funkcji
poniżej

```
@dekorator  
def jakas_funkcja():  
    print("jakis tekst")  
  
jakas_funkcja()
```

WYNIK

Najpierw to
jakis tekst
To na końcu

Przykład 1: Pomiar czasu wykonania

- Chcemy się dowiedzieć które funkcję działają szybko a które nie
- Zamiast kopiować kod do każdej funkcji, mamy wszystko w jednym miejscu.
- Oszczędność czasu + wygoda



```
Czas działania: 0.6003212928771973 sekundy  
Czas działania: 0.0010001659393310547 sekundy
```

```
import time  
  
def pomierzmy_zobaczmy(funkcja):  
    def opakowanie():  
        start = time.time()  
        funkcja()  
        koniec = time.time()  
        print(f"Czas działania: {koniec - start} sekundy")  
    return opakowanie  
  
@pomierzmy_zobaczmy  
def sporo():  
    suma = 0  
    for i in range(10000000):  
        suma += i  
    return suma  
  
@pomierzmy_zobaczmy  
def nie_sporo():  
    suma = 0  
    for i in range(10000):  
        suma += i  
    return suma  
  
sporo()  
nie_sporo()
```

Przykład 2: Walidacja

- Logika walidacji w jednym miejscu

Jeśli funkcja której dajemy dekorator przyjmuje argumenty to opakowanie też musi

```
def waliduj(funkcja):
    def opakowanie(a, b):
        if a < 0 or b < 0:
            raise ValueError("Ujemne pole/objętość xdd?")
        return funkcja(a, b)
    return opakowanie

@waliduj
def pole_prostokata(a, b):
    return a * b

@waliduj
def objetosc_graniastoslupa(szerokosc, wysokosc):
    return szerokosc * wysokosc * szerokosc

pole_prostokata(5, 10)
objetosc_graniastoslupa(-2, 10)
```

Przykład 2: Walidacja

- Logika walidacji w jednym miejscu

Jeśli funkcja której dajemy dekorator przyjmuje argumenty to opakowanie też musi

Ale ogólnie to jest **kiepski** pomysł

```
def waliduj(funkcja):  
    def opakowanie(a, b):  
        if a < 0 or b < 0:  
            raise ValueError("Ujemne pole/objętość xdd?")  
        return funkcja(a, b)  
    return opakowanie  
  
@waliduj  
def pole_prostokata(a, b):  
    return a * b  
  
@waliduj  
def objetosc_graniastoslupa(szerokosc, wysokosc):  
    return szerokosc * wysokosc * szerokosc  
  
pole_prostokata(5, 10)  
objetosc_graniastoslupa(-2, 10)
```

Przykład 3: **kwargs i *args



- Możliwość pobrania wszystkich argumentów funkcji bez konieczności znania ich ilości

```
def cale_zycie_na_plus(funkcja):  
    def opakowanie(*args, **kwargs):  
  
        for arg in args:  
            if arg <= 0:  
                raise ValueError(f"{arg} jest ujemny, i to źle")  
  
        for nazwa, wartosc in kwargs.items():  
            if wartosc <= 0:  
                raise ValueError(f"{nazwa} ma ujemną wartość, i to też źle")  
  
        return funkcja(*args, **kwargs)  
  
    return opakowanie
```



Przykład 4: potrójne zagnieżdzenie

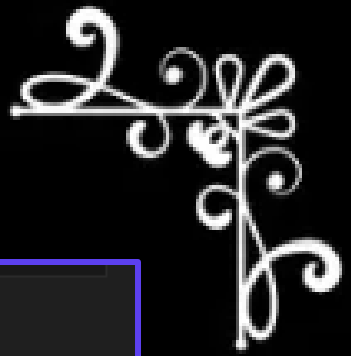
- Dekorator też może przyjmować argumenty

```
def powtorz(ile_razy):  
    def dekorator(funkcja):  
        def opakowanie(*args, **kwargs):  
            for i in range(ile_razy):  
                funkcja(*args, **kwargs)  
            return opakowanie  
        return dekorator
```

```
@powtorz(2)  
def zdzichomat(zdzicho):  
    print(f"Zdzicho mówi: {zdzicho}!")
```

```
@powtorz(3)  
def ben():  
    print("yes")  
  
zdzichomat("Cicho")  
ben()
```

```
Zdzicho mówi: Cicho!  
Zdzicho mówi: Cicho!  
yes  
yes  
yes
```



Ale teraz już na poważnie kochani



To co było do tej pory było
co najwyżej mierne



Były znaki



Fragment z prezentacji mgr inż.
Mirosława Ossyska

Fragment z laba mgr inż.
Konrada Zaworskiego

Flask-Login:

➤ **Opis:** Najważniejsze rozszerzenie. Zarządza sesją użytkownika (zapamiętuje, że użytkownik jest zalogowany po przeładowaniu strony). Dostarcza helpery do logowania, wylogowywania i sprawdzania statusu użytkownika.

➤ Kluczowe elementy:

- **UserMixin:** Klasa bazowa dla modelu użytkownika, implementująca wymagane metody (np. `is_authenticated`).
- `login_user()`: Funkcja logująca użytkownika (tworzy sesję).
- `logout_user()`: Funkcja wylogowująca (czyści sesję).
- `@login_required`: **Dekorator** chroniący widoki (endpointy) przed niezalogowanymi użytkownikami.
- `current_user`: Proxy do obiektu aktualnie zalogowanego użytkownika.

- **eventy własne** - wymyślane przez Ciebie na potrzeby aplikacji, np. `ping`, `add_message`, `new_message`.

Biblioteka Flask-SocketIO dostarcza nam **dekorator** `@socketio.on()`, dzięki któremu możemy oznaczyć funkcje, które będą wykonywane podczas określonych zdarzeń - **zajrzyj do pliku `ws.py` i zauważ, że jest tam funkcja reagująca na zdarzenie `connect` (wbudowane).**

Dekoratory w "sieciówce"

- W frameworkach takich jak **Flask** czy **FastAPI**, dekoratory są sercem architektury.
- Routing: Mapowanie URL na kod
- Middleware: Autoryzacja i sesje
- Serializacja: Konwersja do JSON

Tak, tak to też jest dekorator



```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def strona_glowna():
    return "Jeśli to czytasz, to słabo"

@app.route("/jakis_endpoint_1")
def kontakt():
    return "cokolwiek"

@app.route("/jakis_endpoint_2")
def cennik():
    return "też cokolwiek"

if __name__ == "__main__":
    app.run()
```

Jak działa @app.route?

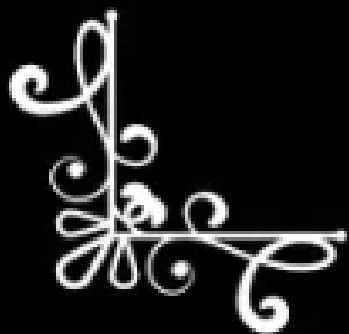


W przeciwieństwie do prostych dekoratorów, `@app.route` służy do **rejestracji**.

1. Wykonuje się w momencie importu skryptu.
2. Nie zmienia funkcji, ale dodaje ją do słownika w obiekcie app.
3. Gdy przychodzi żądanie HTTP na /stronagłowna, Flask zagląda do słownika, bierze funkcję kontakt i ją wywołuje

```
class Flask:
    def __init__(self):
        self.mapa_sciezek = {}

    def route(self, sciezka):
        def dekorator(funkcja):
            self.mapa_sciezek[sciezka] = funkcja
            return funkcja
        return dekorator
```



Inne dekoratory we Flasku



`@login_required` – sprawdza czy użytkownik jest zalogowany

`@app.errorhandler` - rejestruje funkcję jako handler dla konkretnego kodu http

`@app.before_request` – wykonuje się przed każdym żądaniem (interceptor)

Czasem trzeba sobie coś samemu napisać: flask-login nie ma dekoratora pod admina na przykład



```
from functools import wraps
from flask import abort

def admin_required(f):
    @wraps(f) # Dekorator zachowujący informacje o funkcji oryginalnej f
    def decorated_function(*args, **kwargs):
        # Najpierw sprawdzamy czy zalogowany, potem czy ma rolę admin
        if not current_user.is_authenticated or current_user.role != 'admin':
            abort(403) # Błąd 403 Forbidden
        return f(*args, **kwargs)
    return decorated_function

@app.route('/admin_panel')
@login_required # Najpierw wymuś logowanie
@admin_required # Potem wymuś rolę
def admin_panel():
    # Dzięki @wraps zachowujemy nazwę i docstring oryginalnej funkcji admin_panel
    return "To widzi tylko Administrator."
```

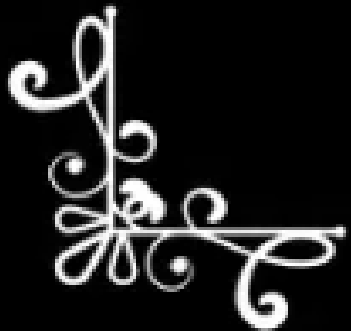

A co tam u Javy?



Java
(Adnotacje)



Python
(Dekoratory)



Porównanie

Python - Flask (Dekorator)

- To **kod wykonujący się**. Bezpośrednio podmienia funkcję na wrapper podczas ładowania modułu.

Java - SpringBoot (Adnotacja)

- To **metadane**. Sama adnotacja nic nie robi – musi zostać odczytana przez Refleksję (np. Spring Proxy).

```
@RestController
@RequestMapping("/users") // Adnotacja na klasie
public class UserController {

    @GetMapping("/{id}") // Adnotacja na metodzie
    public User getUser(@PathVariable String id) {
        // logika
    }
}
```


Metaprogamowanie

(Nie chodzi o mete)



Czym jest Metaprogramowanie?

- To technika, w której programy traktują inne programy jako dane. Pozwala to na:
- Modyfikację kodu w czasie wykonywania
- Automatyczne generowanie kodu
- Tworzenie potężnych abstrakcji (np. ORM)



Zarówno **Dekoratory** w Pythonie jak i **Adnotację** w Javie to przykłady Metaprogramowania

```
class MojaWlasnosc:
    def __get__(self, obj, typ):
        print("Ktoś czyta wartość!")
        return obj._wartosc

    def __set__(self, obj, wartosc):
        print("Ktoś ustawia wartość!")
        obj._wartosc = wartosc

class Klasa:
    atrybut = MojaWlasnosc()
```

Inne przykłady metaprogramowania w Pythonie to **Deskryptory** i **Metaklasy**

ORM-y

- W systemach bazodanowych metaklasy są używane do transformacji klas w tabele SQL.
- Metaklasa przechwytuje moment tworzenia klasy, skanuje jej pola i **generuje strukturę bazy danych**.
- Python – Django
- Java - Hilbernate



Jeszcze troszkę (Metaklasy)



Metaklasa to "klasa,
która tworzy inne klasy"



```
class MojaMeta(type):
    def __new__(cls, nazwa, klasy_bazowe, atrybuty):
        print(f"Tworzę klasę: {nazwa}")
        print(f"  Dziedziczy po: {klasy_bazowe}")
        print(f"  Ma atrybuty: {list(atrybuty.keys())}")

        atrybuty["wersja"] = 1
        atrybuty["data_utworzenia"] = "2026-05-08"

        return super().__new__(cls, nazwa, klasy_bazowe, atrybuty)

class MojaKlasa(metaclass=MojaMeta):
    x = 10
    def metoda(self):
        pass

print(MojaKlasa.wersja)
print(MojaKlasa.data_utworzenia)
```



Dekoratory saq git

Metaprogramowanie też jest git

Dziękujemy za uwagę

